Spring Boot + Database — Lộ trình 1 tháng (học kèm AI)

Nov 2, 2026 · cập nhật Nov 3, 2026 · @thien.lv · tuần 3–6 của Lộ trình Java Backend 12 tuần

Tổng quan

Lộ trình chạy từ thứ Tư 04/11 đến thứ Sáu 04/12/2026: **4 tuần, 23 buổi** (buổi 1 đến buổi 23, mỗi buổi có ngày cụ thể), học từ thứ Hai đến thứ Sáu, 3–4 giờ/buổi. Thứ Bảy, Chủ nhật nghỉ; nếu rảnh thì chỉ dùng để bù chậm hoặc làm thêm bài SQL. File này nối tiếp phần Java Core (xong 03/11) và kết thúc ngay trước giai đoạn dự án (bắt đầu 07/12).

Mục tiêu: hiểu Spring đủ sâu để **tự viết được khi không có AI** và **nhận ra khi AI viết sai**. Cả tháng chỉ làm **một Student Management API** phát triển dần thay vì nhiều project CRUD riêng lẻ. Database học ngay trong tháng này vì AI dựng phần kết nối rất nhanh; thời gian dồn cho phần khó: quan hệ, transaction, N+1, xử lý đồng thời.

Lịch 4 tuần

Tuần	Thời gian	Spring	DB song song	Dùng AI
3	04/11 – 13/11 8 buổi	HTTP/REST, IoC/DI, API đầu tiên, vòng đời request	SQL cơ bản, ràng buộc, khóa ngoại, JOIN; schema 4 bảng	Không dùng AI, tự gõ tay cả tuần
4	16/11 – 20/11 5 buổi	Validation, xử lý lỗi, MapStruct, test MockMvc, cài AI agent	GROUP BY, subquery, CTE, ràng buộc DB vs validation, chuẩn hóa	AI viết phần lặp, bạn review
5	23/11 – 27/11 5 buổi	PostgreSQL, JPA, quan hệ 1-n và n-n, JPQL	Kiểu dữ liệu Java ↔ PostgreSQL, index, `EXPLAIN`, window function	AI dựng docker-compose; quan hệ và SQL tự làm
6	30/11 – 04/12 5 buổi	Phân trang, Flyway, transaction, N+1, khóa dữ liệu	Phân trang, transaction bằng SQL, isolation level, `FOR UPDATE`	AI sinh script Flyway; transaction, N+1, locking tự làm

Quy tắc trong tháng

- **Quy tắc vàng: không commit dòng code nào mà bạn không giải thích được.** Khi AI viết, hỏi lại "vì sao" cho đến khi hiểu, hoặc xóa đi tự viết.
- **Tuần 3 không mở AI** để viết code. Kẹt quá 30 phút thì chỉ nhờ AI *giải thích* thông báo lỗi, không xin code sửa sẵn.
- **Nhật ký review AI** (từ tuần 4): mỗi lần AI viết sai (N+1, quên `@Transactional`, nuốt exception, dùng API cũ của Spring Boot 3…) thì ghi một dòng. Mục tiêu cuối tháng: ít nhất 10 lỗi.
- **DB song song mỗi buổi 45–60 phút, tự gõ, không AI** (trừ việc sinh dữ liệu mẫu). Chi tiết ghi trong từng buổi.
- **30 phút cuối buổi thứ Sáu:** trả lời câu hỏi tự kiểm tra của tuần, cập nhật nhật ký, đẩy code lên GitHub và gắn tag `week-N`.

Lịch 23 buổi (bản tóm tắt để đánh dấu)

Thứ Bảy, Chủ nhật không có buổi học. Chi tiết từng buổi ở phần "Kế hoạch từng buổi" ngay sau.

	Buổi	Ngày	Spring	DB song song
Tuần 3				
☐	1	T4 04/11	HTTP và REST	Cài PostgreSQL, bảng đầu tiên
☐	2	T5 05/11	IoC và Dependency Injection	SELECT, WHERE, ORDER BY
☐	3	T6 06/11	Project Spring Web đầu tiên	Ràng buộc dữ liệu
☐	4	T2 09/11	@RequestBody, ResponseEntity, status code	UPDATE, DELETE
☐	5	T3 10/11	Controller → Service → Repository, DTO	Khóa ngoại, quan hệ 1-n
☐	6	T4 11/11	Vòng đời một request	Quan hệ n-n, file schema.sql
☐	7	T5 12/11	Ôn tập: viết lại không nhìn	INNER JOIN, LEFT JOIN
☐	8	T6 13/11	Vá chỗ hổng	Ôn DB, dữ liệu mẫu
Tuần 4				
☐	9	T2 16/11	Validation	GROUP BY, hàm tổng hợp
☐	10	T3 17/11	Xử lý lỗi chuẩn (tự viết)	Ràng buộc DB và validation
☐	11	T4 18/11	Lombok, MapStruct, logging, profile	Subquery, EXISTS, CTE
☐	12	T5 19/11	Test controller với MockMvc	Chuẩn hóa dữ liệu
☐	13	T6 20/11	Làm việc với AI agent	Luyện tập SQL
Tuần 5				
☐	14	T2 23/11	PostgreSQL + Spring Data JPA	Kiểu dữ liệu Java ↔ PostgreSQL
☐	15	T3 24/11	@ManyToOne, @OneToMany	Index cho khóa ngoại, EXPLAIN
☐	16	T4 25/11	Quan hệ n-n qua Enrollment	Index B-tree, EXPLAIN ANALYZE
☐	17	T5 26/11	JPQL, @Query, native query	Window function
☐	18	T6 27/11	Test repository với @DataJpaTest	Luyện tập SQL
Tuần 6				
☐	19	T2 30/11	Phân trang và Flyway	Phân trang trong SQL
☐	20	T3 01/12	@Transactional	Transaction bằng SQL
☐	21	T4 02/12	Lỗi N+1	N+1 nhìn từ phía DB
☐	22	T5 03/12	Ghi đồng thời: optimistic locking	Isolation level, SELECT … FOR UPDATE
☐	23	T6 04/12	Tổng hợp và tự kiểm tra	Thiết kế bảng cho dự án

Bắt đầu Spring Boot từ đâu

Đừng bắt đầu bằng việc tạo project Spring. Spring Boot về cơ bản là lớp bọc quanh HTTP cộng với một "nhà máy" tự tạo và lắp ráp object. Hiểu hai thứ đó trước thì phần còn lại dễ hơn nhiều. Thứ tự đi vào:

1. **HTTP trước, Spring sau** (buổi 1): request/response, method, status code, JSON. Bạn đã thấy chúng ở frontend, giờ nhìn từ phía server.

2. **Dependency Injection bằng Java thuần trước** (nửa đầu buổi 2): viết `StudentService` tự `new StudentRepository()` bên trong → sửa thành nhận repository qua constructor → tự viết hàm `main` tạo và lắp các object với nhau. Đó chính là việc Spring làm hộ bạn. Nửa sau buổi mới chuyển sang `@Service`, `@Repository`.
3. **Project Spring đầu tiên chỉ có 1 dependency** (buổi 3): Spring Web. Mỗi tuần chỉ thêm dependency khi cần, để biết mỗi cái làm gì.
4. **Mỗi khái niệm đi qua 3 bước:** đọc guide/tài liệu 20–30 phút → tự gõ một ví dụ nhỏ → áp vào Student API.

Chuẩn bị công cụ (tối 03/11, trước buổi 1)

Công cụ	Ghi chú
JDK 21 hoặc 25 (bản LTS)	Dùng lại JDK của phần Java Core; kiểm tra bằng `java -version`
IntelliJ IDEA	Bản miễn phí đủ dùng cho cả giai đoạn 2
Postman hoặc Bruno	Gọi thử API; lưu request thành collection theo từng tuần
Git + repo GitHub `student-management-api`	Tạo từ buổi 3; cuối mỗi tuần gắn tag `week-3`, `week-4`… để nhìn lại tiến độ
PostgreSQL + DBeaver	Cần ngay từ buổi 1 cho phần DB song song (có thể chạy PostgreSQL bằng Docker)
Docker Desktop	Cần từ tuần 5; cài sẵn để không mất buổi vì lỗi cài đặt

Tạo project trên start.spring.io (buổi 3 · Thứ Sáu 06/11)

Mục	Chọn
Project / Language	Maven (giống bài 33) / Java
Spring Boot	Bản ổn định mới nhất — không chọn bản có chữ SNAPSHOT, M hoặc RC. Đầu tháng 11/2026 là dòng 4.1.x; 4.2.0 đang ở giai đoạn RC, nếu lúc tạo project đã có bản 4.2.0 chính thức thì chọn bản đó.
Group / Artifact	`com.thienlv` / `student-api`
Packaging / Java	Jar / 21 (hoặc 25)
Dependencies	Tuần 3: **Spring Web**. Tuần 4: thêm Validation, Lombok (MapStruct thêm tay vào `pom.xml`). Tuần 5: Spring Data JPA, PostgreSQL Driver. Tuần 6: Flyway Migration.

Lưu ý phiên bản: Spring Boot 4 dùng Jackson 3 và tách lại các module test, trong khi nhiều bài Baeldung và code AI sinh ra vẫn viết theo Spring Boot 3. Khi import báo lỗi hoặc annotation không còn (ví dụ `@MockBean` đã được thay bằng `@MockitoBean`), tra Release Notes / Migration Guide của Spring Boot thay vì hạ phiên bản. Đây cũng là một loại lỗi AI nên ghi vào nhật ký review.

Cấu trúc package gợi ý

```
com.thienlv.studentapi
├── StudentApiApplication.java
├── controller/   StudentController, ClassRoomController
├── service/      StudentService, ClassRoomService
├── repository/   StudentRepository                    (Map ở tuần 3, JPA từ tuần 5)
├── dto/          StudentRequest, StudentResponse (record)
├── entity/       Student, ClassRoom, Course, Enrollment  (từ tuần 5)
└── exception/    NotFoundException, GlobalExceptionHandler (từ tuần 4)
```

Chia theo tầng để thấy rõ luồng Controller → Service → Repository. Sang dự án ở tuần 7 có thể chuyển sang chia theo tính năng.

Khung một buổi 3–4 giờ

Thời lượng	Việc
20–30 phút	Đọc tài liệu của chủ đề (ghi ngay trong từng buổi)
1,5–2 giờ	Spring: ví dụ nhỏ trước, rồi áp vào Student API; commit sau mỗi phần chạy được
45–60 phút	DB song song (phần "DB" của buổi; tự gõ, không AI)
10–15 phút	Tự giải thích 1 khái niệm bằng lời; ghi lỗi gặp phải và lỗi AI vào nhật ký

Student API lớn dần qua 4 tuần

Tuần	Endpoint / tính năng mới	Kết quả cuối tuần
3	`GET /api/students` (lọc `?name=&minAge=&maxAge=`), `GET /api/students/{id}`, `POST /api/students`, `PUT /api/students/{id}`, `DELETE /api/students/{id}`	CRUD 3 tầng, lưu trong Map, đúng status 200/201/204/404
4	Trả 400 khi dữ liệu sai; mọi lỗi trả cùng format `ProblemDetail`; CRUD `/api/classes` (AI làm theo file quy ước)	Có test MockMvc cho `StudentController`
5	Lưu vào PostgreSQL; `GET /api/classes/{id}/students`; CRUD `/api/courses`; `POST /api/enrollments` (kèm điểm); `GET /api/courses/{id}/average-score`; `GET /api/students/without-courses`; (tùy chọn) top 3 sinh viên mỗi khóa bằng native query	Có ERD; test `@DataJpaTest` cho repository
6	`GET /api/students?page=&size=&sort=`; `PUT /api/students/{id}/class/{classId}` (chuyển lớp + ghi lịch sử, có transaction); `POST /api/classes/{id}/students` (lớp có sĩ số tối đa, 2 request cùng lúc không làm vượt sĩ số, trả 409); `GET /api/classes` kèm sinh viên không bị N+1; Flyway `V1`–`V4`	Tag `week-6`; đạt tiêu chí chuyển giai đoạn

Học DB song song với Spring

Mỗi buổi dành **45–60 phút cho DB**, ngay sau phần Spring. Phần DB không phải một môn tách rời: chủ đề DB mỗi buổi được chọn để khớp với phần Spring cùng ngày hoặc đi trước nó một chút, để khi Spring "làm hộ" thì bạn đã biết bên dưới nó là câu SQL nào.

Khi Spring học...	...thì DB học
API lọc, CRUD (tuần 3)	Câu SQL tương ứng mỗi endpoint: `SELECT … WHERE`, `INSERT`, `UPDATE`, `DELETE`
Validation, xử lý lỗi (tuần 4)	Ràng buộc trong DB (`UNIQUE`, `CHECK`, khóa ngoại) và lỗi DB trả về khi vi phạm
JPA, quan hệ (tuần 5)	SQL Hibernate sinh ra, index cho khóa ngoại, `EXPLAIN`
`@Transactional`, N+1, `@Version` (tuần 6)	`BEGIN/COMMIT/ROLLBACK`, JOIN thay cho N câu nhỏ, isolation level, `SELECT … FOR UPDATE`

- **Dùng chung một schema:** students, classes, courses, enrollments — đúng các bảng của Student API. Đến tuần 5, JPA chỉ việc map lên schema bạn đã tự thiết kế; đến tuần 6, file `schema.sql` của bạn trở thành `V1__init.sql`

của Flyway.

- **Lưu bài trong repo:** thư mục `sql/week-3/`, `sql/week-4/`... mỗi buổi một file, có ghi chú kết quả.
- **Thực hành trong DBeaver**, tắt auto-commit khi học transaction; mở 2 cửa sổ để mô phỏng 2 người dùng cùng lúc.

Kế hoạch từng buổi

Thời lượng chỉ để tham khảo; tổng mỗi buổi khoảng 3–4 giờ. Mỗi mục "Xong khi" là điều kiện để coi buổi đó hoàn thành — chưa đạt thì bù vào buổi sau hoặc cuối tuần.

Tuần 3 — HTTP, Spring Core, API đầu tiên

04/11 – 13/11 · 8 buổi

Tự gõ tay, **không dùng AI**. Cuối tuần: Student API CRUD 3 tầng (lưu trong Map) và file `schema.sql` 4 bảng.

Buổi 1 **Thứ Tư 04/11/2026** HTTP và REST

Spring ≈ 2 giờ 20	• **30 phút** — Nếu tối 03/11 chưa cài: JDK, IntelliJ, Postman, Git (bảng "Chuẩn bị công cụ"). • **40 phút** — Đọc MDN: HTTP: Overview, HTTP request methods, HTTP response status codes. Ghi chú: GET/POST/PUT/PATCH/DELETE khác nhau thế nào; "idempotent" là gì; khi nào dùng 200, 201, 204, 400, 401, 403, 404, 409, 500. • **30 phút** — DevTools → Network trên một trang thật: chọn 5 request, ghi method, URL, status, header `Content-Type`, body. • **40 phút** — Tự viết bảng API cho bài quản lý sinh viên (URL, method, request, response, status thành công và lỗi) vào file `docs/api.md`. Viết xong mới đối chiếu với bảng "Student API lớn dần".
DB 50 phút	Cài PostgreSQL (Docker hoặc bản cài) và DBeaver; tạo database `student_db`. Kiểu dữ liệu cơ bản: `bigint`, `varchar/text`, `date`, `timestamp`, `numeric`, `boolean`. Tạo bảng `students` (id, full_name, email, birth_date, created_at), `INSERT` 10 dòng bằng tay. Lưu vào `sql/week-3/01-create.sql`. Đọc: PostgreSQL Tutorial, chương 2 mục 2.1–2.5.
Cuối buổi 10–15 phút	Tự giải thích bằng lời: "Khi bấm nút Lưu trên frontend, điều gì đi qua mạng và server trả về gì?"
Xong khi	☐ Bảng API có ít nhất 5 endpoint, mỗi cái có status thành công và lỗi ☐ Bảng students có 10 dòng

Buổi 2 **Thứ Năm 05/11/2026** IoC và Dependency Injection

Spring
≈ 2 giờ

- **25 phút** — Đọc Spring Framework reference → Core Technologies → The IoC Container: Introduction, Container Overview, Dependency Injection (phần constructor-based).
- **40 phút** — **DI bằng Java thuần** (project Maven thường, chưa có Spring): viết `Student` (record), `StudentRepository` lưu trong `Map`, `StudentService` tự `new StudentRepository()` bên trong. Thấy chỗ khó: muốn đổi repository hay viết test thì phải sửa service. Sửa thành nhận repository qua constructor, rồi viết hàm `main` tự tạo và lắp các object với nhau.
- **40 phút** — **Chuyển sang Spring**: tạo project tạm `di-demo` trên start.spring.io không chọn dependency nào. Đánh `@Repository`, `@Service`, dùng constructor injection; in danh sách sinh viên bằng một `CommandLineRunner`. Thử tạo 2 class cùng implement một interface để gặp lỗi `NoUniqueBeanDefinitionException`, sửa bằng `@Primary` hoặc `@Qualifier`.
- **15 phút** — Vẽ sơ đồ: Spring quét `@Component` → tạo bean → tiêm qua constructor. Tự trả lời: vì sao constructor injection tốt hơn `@Autowired` trên field?

DB
50 phút

`SELECT`, `WHERE`, `AND/OR`, `IN`, `BETWEEN`, `LIKE/ILIKE`, `IS NULL`, `ORDER BY`, `LIMIT/OFFSET`. Viết 10 câu truy vấn trên `students`, trong đó có tìm theo tên (`ILIKE`) và theo khoảng tuổi (tính từ `birth_date`) — chính là API buổi 3 sẽ làm. Lưu `sql/week-3/02-select.sql`.
Đọc: SQLBolt bài 1–5.

Cuối buổi
10–15 phút

Ghi vào sổ tay 3 câu: bean là gì, ai tạo bean, vì sao dùng constructor injection.

Xong khi

- ☐ Bản Java thuần chạy được với service nhận repository qua constructor
- ☐ Bản Spring in ra danh sách sinh viên qua bean
- ☐ 10 câu SELECT chạy đúng

Buổi 3 **Thứ Sáu 06/11/2026** Project Spring Web đầu tiên

Spring
≈ 2 giờ

- **20 phút** — Spring guide: Building a RESTful Web Service — đọc và chạy thử.
- **20 phút** — Tạo project `student-api` trên start.spring.io theo bảng "Tạo project", chỉ chọn **Spring Web**. Mở bằng IntelliJ, chạy được. Tạo repo GitHub `student-management-api`, commit đầu tiên.
- **20 phút** — Đọc hiểu cấu trúc project: `pom.xml`, class `StudentApiApplication`, đổi `application.properties` thành `application.yml`, thử đổi cổng sang 8081.
- **60 phút** — `@RestController`, `@GetMapping`, `@PathVariable`, `@RequestParam`: `GET /hello?name=`; `GET /api/students` (danh sách tạm viết cứng trong code); `GET /api/students/{id}`; lọc `GET /api/students?name=&minAge=&maxAge=`. Gọi thử bằng Postman.

DB
50 phút

`PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`, cột tự tăng `GENERATED ALWAYS AS IDENTITY`. Tạo lại bảng `students` có đủ ràng buộc (`UNIQUE` email, `CHECK` ngày sinh trước hôm nay). Cố tình `INSERT` sai 5 kiểu, chép lại thông báo lỗi. Lưu `sql/week-3/03-constraints.sql`.
Đọc: SQLBolt bài 16; PostgreSQL docs → Constraints.

Cuối buổi
10–15 phút

Đẩy code lên GitHub. Ôn nhanh 3 buổi đầu: tự giải thích lại DI trong 2 phút.

Xong khi

- ☐ Repo GitHub có commit đầu tiên
- ☐ 4 endpoint GET chạy được trên Postman
- ☐ Bảng students có đủ ràng buộc

Buổi 4 **Thứ Hai 09/11/2026** @RequestBody, ResponseEntity, status code

Spring ≈ 2 giờ	• **20 phút** — Đọc Spring Framework reference → Web on Servlet Stack → Annotated Controllers: mục `@RequestBody` và `ResponseEntity`. • **40 phút** — `POST /api/students` nhận JSON bằng `@RequestBody` (record `StudentRequest`). Quan sát Jackson chuyển JSON ↔ object; thử gửi JSON sai kiểu (tuổi là chữ) xem server trả gì. • **40 phút** — Dùng `ResponseEntity`: POST trả **201** kèm header `Location`; `GET /{id}` không có trả **404**; `PUT /{id}` trả 200; `DELETE /{id}` trả **204**. • **20 phút** — Lưu tất cả request thành Postman collection "Week 3".
DB 50 phút	`UPDATE`, `DELETE`, `RETURNING`; tập thói quen chạy thử trong `BEGIN … ROLLBACK` trước khi chạy thật để tránh quên `WHERE`. Thêm vào `docs/api.md` một cột "SQL tương ứng" cho từng endpoint. Đọc: SQLBolt bài 13–15.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ POST/PUT/DELETE trả đúng 201/200/204, không tìm thấy trả 404 ☐ Mỗi endpoint trong docs/api.md có câu SQL tương ứng

Buổi 5 **Thứ Ba 10/11/2026** Controller → Service → Repository, DTO

Spring ≈ 2 giờ	• **15 phút** — Xem lại cấu trúc package gợi ý; nhớ vai trò từng tầng. • **90 phút** — Tách code thành 3 tầng: `StudentController` chỉ nhận/trả HTTP; `StudentService` chứa logic (kiểm tra trùng email, tính tuổi); `StudentRepository` lưu `Map<Long, Student>` và sinh id bằng `AtomicLong`. DTO `StudentRequest`/`StudentResponse` dạng record, chuyển đổi thủ công trong service. • **15 phút** — Chạy lại toàn bộ Postman collection, không lỗi.
DB 50 phút	`FOREIGN KEY`, quan hệ 1-n, `ON DELETE RESTRICT` / `CASCADE` / `SET NULL`. Tạo bảng `classes`, thêm cột `students.class_id`; thử xóa một lớp còn sinh viên với từng lựa chọn và ghi kết quả. Đọc: PostgreSQL Tutorial mục 3.3 Foreign Keys.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ CRUD sinh viên chạy qua đủ 3 tầng, controller không chứa logic ☐ Bảng classes có khóa ngoại, biết khác nhau giữa 3 kiểu ON DELETE

Buổi 6 **Thứ Tư 11/11/2026** Vòng đời một request

Spring ≈ 1 giờ 50	• **25 phút** — Đọc Spring Framework reference → Web on Servlet Stack → DispatcherServlet (phần mở đầu và Processing). • **60 phút** — Viết một filter đơn giản (`OncePerRequestFilter`) log thời gian xử lý mỗi request. Đặt breakpoint ở filter, controller, service; debug đi theo một request; trong stack trace của IntelliJ tìm `DispatcherServlet.doDispatch` và chỗ Jackson ghi JSON (`HttpMessageConverter`). • **25 phút** — Vẽ sơ đồ vòng đời: Tomcat → Filter → DispatcherServlet → HandlerMapping → Controller → Service → Repository → HttpMessageConverter → response.
DB 55 phút	Quan hệ n-n qua bảng trung gian: `courses`, `enrollments` (student_id, course_id, score) với `UNIQUE (student_id, course_id)`. Vẽ ERD 4 bảng (giấy hoặc ER Diagram của DBeaver). Gom tất cả thành `sql/schema.sql` chạy lại được từ đầu (`DROP TABLE IF EXISTS` rồi `CREATE`).
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ Có sơ đồ vòng đời request và giải thích được bằng lời ☐ schema.sql 4 bảng chạy lại từ đầu không lỗi

Buổi 7 **Thứ Năm 12/11/2026** Ôn tập: viết lại không nhìn

Spring 2 giờ	• **2 giờ** — Tạo project mới `student-api-rewrite` (xóa sau). Viết lại Student API CRUD 3 tầng + DTO + đúng status code trong 2 giờ, **không nhìn code cũ, không AI**. Mỗi lần phải dừng lại tra cứu, ghi một dòng vào danh sách "chỗ hổng".
DB 50 phút	`INNER JOIN`, `LEFT JOIN`: sinh viên kèm tên lớp; sinh viên chưa có lớp; các khóa học của một sinh viên; mỗi khóa có những ai. Lưu `sql/week-3/07-join.sql`. Đọc: SQLBolt bài 6–8.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ Bản viết lại chạy được (chưa đủ cũng được) ☐ Có danh sách "chỗ hổng" cho buổi 8 ☐ 4 câu JOIN chạy đúng

Buổi 8 **Thứ Sáu 13/11/2026** Vá chỗ hổng

Spring ≈ 1 giờ 40	• **60 phút** — Học lại từng mục trong danh sách "chỗ hổng" của buổi 7. • **20 phút** — Tự giải thích bằng lời IoC/DI và vòng đời request (có thể tự ghi âm 3 phút rồi nghe lại). • **20 phút** — Dọn code, viết `README.md` ngắn: cách chạy, danh sách endpoint.
DB 50 phút	Viết lại `schema.sql` từ đầu không nhìn. Nhờ AI sinh khoảng 50 sinh viên, 5 lớp, 6 khóa và các lượt đăng ký mẫu (vùng giao AI; kiểm tra dữ liệu có thỏa ràng buộc). Viết 10 câu SELECT/JOIN trên dữ liệu đó.
Cuối buổi 30 phút	**30 phút tự kiểm tra tuần 3** (trả lời bằng lời, không mở tài liệu): PUT và PATCH khác nhau thế nào, khi nào trả 201, 204, 409? · Bean là gì, ai tạo, tạo lúc nào, mặc định sống bao lâu? · Vì sao constructor injection tốt hơn field injection? · `@Controller` khác `@RestController` ở đâu? · Một request đi qua những bước nào? · `ON DELETE CASCADE` và `RESTRICT` khác nhau thế nào? · `INNER JOIN` và `LEFT JOIN` khác nhau ở đâu? Sau đó đẩy code, gắn tag `week-3`.
Xong khi	☐ Trả lời được toàn bộ câu tự kiểm tra tuần 3 ☐ Repo có tag `week-3`

Tuần 4 — Validation, xử lý lỗi, test, làm việc với AI agent

16/11 – 20/11 · 5 buổi

Bắt đầu cho AI viết phần lặp, bạn review và ghi nhật ký. Cuối tuần: mọi lỗi cùng format, có test controller, có file quy ước cho AI.

Buổi 9 **Thứ Hai 16/11/2026** Validation

Spring 2 giờ	• **20 phút** — Đọc Spring Boot reference → mục Validation; lướt danh sách annotation của Jakarta Bean Validation. • **15 phút** — Thêm dependency Validation. Tạo file `docs/ai-review-log.md` (nhật ký review AI). • **20 phút** — Tự liệt kê trước các trường hợp dữ liệu sai: tên rỗng hoặc toàn khoảng trắng, email sai định dạng, ngày sinh ở tương lai, tên quá dài… • **45 phút** — Nhờ AI thêm validation cho `StudentRequest` (`@NotBlank`, `@Size`, `@Email`, `@Past`…) và `@Valid` ở controller. Đối chiếu với danh sách của bạn: AI bỏ sót case nào? Ghi vào nhật ký. • **20 phút** — Postman thử từng case sai, xem response 400 mặc định trông thế nào (buổi sau sẽ chuẩn hóa).
DB 50 phút	`GROUP BY`, `COUNT/AVG/MIN/MAX/SUM`, `HAVING`: sĩ số mỗi lớp; điểm trung bình mỗi khóa; các lớp có từ 10 sinh viên trở lên. Đọc: SQLBolt bài 9–12.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ Mọi request sai trả 400 ☐ Nhật ký review AI có dòng đầu tiên ☐ 3 câu GROUP BY chạy đúng

Buổi 10 **Thứ Ba 17/11/2026** Xử lý lỗi chuẩn (tự viết)

Spring ≈ 2 giờ	• **20 phút** — Đọc Spring Framework reference → Web on Servlet Stack → Error Responses (`ProblemDetail`). • **80 phút** — **Tự viết, không AI**: `NotFoundException`, `DuplicateEmailException`; `GlobalExceptionHandler` với `@RestControllerAdvice`: không tìm thấy → 404; `MethodArgumentNotValidException` → 400 kèm danh sách field lỗi; trùng email → 409; lỗi không lường trước → 500 và không lộ stack trace. Tất cả trả `ProblemDetail` cùng một format. • **15 phút** — Postman kiểm lại mọi case lỗi.
DB 50 phút	Vi phạm ràng buộc thì DB trả lỗi gì: mã SQLSTATE 23505 (unique), 23503 (khóa ngoại), 23514 (check). Lập bảng: mỗi validation ở Java tương ứng với ràng buộc nào trong DB; giải thích vì sao vẫn cần `UNIQUE` dù service đã kiểm tra trùng email (gợi ý: 2 request cùng lúc).
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ Mọi lỗi (404, 400, 409, 500) trả cùng format ProblemDetail ☐ Có bảng đối chiếu validation ↔ ràng buộc DB

Buổi 11 **Thứ Tư 18/11/2026** Lombok, MapStruct, logging, profile

Spring ≈ 1 giờ 50	• **15 phút** — MapStruct — đọc trang Getting started. • **40 phút** — Thêm Lombok và MapStruct. Nhờ AI chuyển mapping thủ công sang `StudentMapper`; build rồi mở `target/generated-sources` đọc code MapStruct sinh ra. Kiểm tra AI có cấu hình đúng annotation processor cho cả Lombok và MapStruct không (lỗi hay gặp) — ghi nhật ký. • **30 phút** — Logging SLF4J: log mức INFO khi tạo/xóa sinh viên, không log dữ liệu nhạy cảm; đổi mức log trong `application.yml`. • **25 phút** — Profile: `application-dev.yml` và `application-prod.yml`; chạy với `--spring.profiles.active=dev`; in ra một giá trị khác nhau giữa 2 profile để kiểm tra.
DB 50 phút	Subquery, `EXISTS`/`NOT EXISTS`, `IN`, CTE (`WITH`). Sinh viên chưa đăng ký khóa nào, viết 3 cách: `LEFT JOIN … IS NULL`, `NOT EXISTS`, `NOT IN`. Rồi tìm "lớp chưa có sinh viên" bằng `NOT IN (SELECT class_id FROM students)` khi có sinh viên chưa xếp lớp (`class_id` NULL) — thấy kết quả rỗng và giải thích vì sao.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ Mapper MapStruct chạy, đọc hiểu code sinh ra ☐ Chạy được 2 profile ☐ Giải thích được bẫy NULL của NOT IN

Buổi 12 **Thứ Năm 19/11/2026** Test controller với MockMvc

Spring ≈ 1 giờ 55	• **20 phút** — Đọc Spring Boot reference → Testing → Testing Spring Boot Applications, mục `@WebMvcTest`. • **25 phút** — Tự liệt kê test case cho StudentController: tạo thành công 201, thiếu tên 400, email sai 400, trùng email 409, id không có 404, xóa 204… • **60 phút** — AI viết code test bằng `@WebMvcTest` + MockMvc, mock service bằng `@MockitoBean`. Bạn chạy, đọc từng test, sửa. Kiểm tra AI có dùng `@MockBean` (đã bỏ ở Spring Boot 4) không — ghi nhật ký. • **10 phút** — `mvn test` chạy xanh.
DB 50 phút	Chuẩn hóa 1NF–3NF ở mức thực dụng; khi nào chấp nhận lặp dữ liệu. Cho một bảng "phẳng" kiểu Excel (sinh viên, lớp, khóa, điểm trên cùng một dòng), tách thành các bảng chuẩn và so với schema của bạn.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ Ít nhất 6 test controller chạy xanh ☐ Bảng phẳng đã được tách đúng

Buổi 13 **Thứ Sáu 20/11/2026** Làm việc với AI agent

Spring ≈ 1 giờ 50	• **20 phút** — Cài Claude Code hoặc Cursor, mở repo. • **30 phút** — Viết file quy ước cho AI (`CLAUDE.md` hoặc rules của Cursor): kiến trúc 3 tầng, cách đặt tên, DTO dạng record, constructor injection, format lỗi ProblemDetail, test bằng `@WebMvcTest`. • **60 phút** — Giao AI làm Class API (CRUD `/api/classes`) theo quy ước. Review từng file trước khi nhận: đúng tầng chưa, status code đúng chưa, có test chưa. Ghi các lỗi bắt được vào nhật ký.
DB 45 phút	5 bài LeetCode SQL 50 mức Easy.
Cuối buổi 30 phút	**30 phút tự kiểm tra tuần 4**: `@Valid` đặt ở đâu, lỗi validation ném exception gì và bắt ở đâu? · `@ExceptionHandler` trong controller khác `@RestControllerAdvice` thế nào? · MapStruct sinh code lúc nào, vì sao không cần reflection? · `@WebMvcTest` tải những bean nào, vì sao phải mock service? · Kể 3 lỗi AI viết sai mà bạn bắt được · Đã có kiểm tra trùng email ở service, vì sao vẫn cần `UNIQUE`? · `WHERE` và `HAVING` khác nhau thế nào? Cập nhật nhật ký, gắn tag `week-4`.
Xong khi	☐ Class API chạy và có test ☐ Có file quy ước cho AI ☐ Repo có tag `week-4`

Tuần 5 — PostgreSQL, JPA, quan hệ

23/11 – 27/11 · 5 buổi

Cuối tuần: API lưu vào PostgreSQL, đủ quan hệ 1-n và n-n, có thống kê bằng JPQL và test repository.

Buổi 14 **Thứ Hai 23/11/2026** PostgreSQL + Spring Data JPA

Spring 2 giờ	• **20 phút** — Spring guide: Accessing Data with JPA. • **20 phút** — Nhờ AI viết `compose.yaml` chạy PostgreSQL (database riêng `student_api`, tách với `student_db` dùng để luyện SQL). Thêm Spring Data JPA và PostgreSQL Driver; cấu hình datasource trong `application-dev.yml`. • **60 phút** — Biến `Student` thành `@Entity` (`@Id`, `@GeneratedValue(strategy = IDENTITY)`, `@Column`); `StudentRepository extends JpaRepository<Student, Long>` thay cho Map. Bật `show-sql` và `format_sql`. Tạm dùng `ddl-auto=update` cho Hibernate tạo bảng (tuần 6 thay bằng Flyway). • **20 phút** — Thay lọc bằng tay bằng query method: `findByFullNameContainingIgnoreCase`, `findByBirthDateBetween`; đọc SQL được sinh ra.
DB 50 phút	Đối chiếu kiểu dữ liệu: `Long` ↔ `bigint`, `LocalDate` ↔ `date`, `BigDecimal` ↔ `numeric`, enum ↔ `varchar`. So bảng do Hibernate tạo với `schema.sql` của bạn, ghi lại các điểm khác (kiểu cột, độ dài, ràng buộc bị thiếu).
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ Dữ liệu vẫn còn sau khi khởi động lại app ☐ Ghi được ít nhất 3 khác biệt giữa bảng Hibernate tạo và schema.sql

Buổi 15 **Thứ Ba 24/11/2026** @ManyToOne, @OneToMany

Spring ≈ 1 giờ 50	• **15 phút** — Vlad Mihalcea: đọc bài về cách map `@OneToMany` và `@ManyToOne`. • **15 phút** — Tự vẽ ERD Student – ClassRoom trước khi code. • **60 phút** — Entity `ClassRoom`; `Student` có `@ManyToOne(fetch = LAZY)` tới lớp; `ClassRoom` có `@OneToMany(mappedBy = "classRoom")`. Endpoint `GET /api/classes/{id}/students`, trả DTO chứ không trả entity (tránh vòng lặp JSON). • **20 phút** — Thử `cascade = CascadeType.ALL` trên ClassRoom rồi xóa một lớp: chuyện gì xảy ra với sinh viên? Ghi lại rồi bỏ cascade.
DB 50 phút	PostgreSQL **không tự tạo index** cho cột khóa ngoại. Sinh thêm vài nghìn sinh viên bằng `generate_series`; chạy `EXPLAIN` câu lọc theo `class_id`, tạo index cho `students.class_id`, so kết quả trước và sau. Đọc: PostgreSQL docs → Using EXPLAIN (phần đầu).
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ GET sinh viên theo lớp chạy, response là DTO ☐ Giải thích được mappedBy và bên nào giữ khóa ngoại ☐ Thấy EXPLAIN đổi sau khi thêm index

Buổi 16 **Thứ Tư 25/11/2026** Quan hệ n-n qua Enrollment

Spring ≈ 1 giờ 45	• **15 phút** — Vlad Mihalcea: đọc bài về many-to-many có cột phụ (dùng entity trung gian). • **75 phút** — Entity `Course` và `Enrollment` (`@ManyToOne` student, `@ManyToOne` course, cột score, unique constraint trên (student_id, course_id)). CRUD `/api/courses` có thể giao AI. **Tự viết** POST `/api/enrollments`: kiểm tra sinh viên và khóa tồn tại (404), không cho đăng ký trùng (409). • **15 phút** — Postman: đăng ký thành công, đăng ký trùng, khóa không tồn tại.
DB 55 phút	Sinh 100.000 dòng `enrollments` bằng `generate_series`. Index nhiều cột `(student_id, course_id)`: thứ tự cột ảnh hưởng thế nào khi lọc chỉ theo course_id? Đọc `EXPLAIN ANALYZE`: Seq Scan và Index Scan. Đọc: PostgreSQL docs → Indexes; Use The Index, Luke chương đầu.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ Đăng ký khóa học chạy; đăng ký trùng trả 409 ☐ Thấy được Index Scan và giải thích thứ tự cột trong index

Buổi 17 **Thứ Năm 26/11/2026** JPQL, @Query, native query

Spring 2 giờ	• **15 phút** — Đọc Spring Data JPA reference → JPA Query Methods → Using `@Query`. • **30 phút** — Viết SQL thuần trong DBeaver trước: điểm trung bình theo khóa; sinh viên chưa đăng ký khóa nào. • **60 phút** — Chuyển thành JPQL bằng `@Query`, trả về record DTO: `GET /api/courses/{id}/average-score`, `GET /api/students/without-courses`. • **15 phút** — Sau phần DB: đưa câu "top 3" vào một native query, nếu còn thời gian.
DB 50 phút	Window function: `ROW_NUMBER`, `RANK`, `AVG(…) OVER (PARTITION BY …)`. Bài: top 3 sinh viên điểm cao nhất mỗi khóa; so điểm mỗi sinh viên với điểm trung bình khóa.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ 2 endpoint thống kê chạy, SQL trong log đúng như bạn nghĩ ☐ Câu top 3 mỗi khóa chạy đúng

Buổi 18 **Thứ Sáu 27/11/2026** Test repository với @DataJpaTest

Spring ≈ 1 giờ 45	• **15 phút** — Đọc Spring Boot reference → Testing → mục Data JPA tests. • **60 phút** — `@DataJpaTest` cho StudentRepository và EnrollmentRepository: query method, `@Query`, đăng ký trùng phải ném exception. Môi trường test: nhờ AI dựng Testcontainers PostgreSQL (bạn review), hoặc tạm trỏ vào PostgreSQL Docker đang chạy với `@AutoConfigureTestDatabase(replace = NONE)`. • **30 phút** — Chạy lại toàn bộ API tuần 3–5 bằng Postman; sửa chỗ hỏng.
DB 45 phút	5 bài LeetCode SQL 50 mức Easy và Medium.
Cuối buổi 30 phút	**30 phút tự kiểm tra tuần 5**: Entity khác DTO thế nào, vì sao không trả entity ra API? · `mappedBy` nghĩa là gì, bên nào giữ khóa ngoại? · Vì sao quan hệ n-n có cột điểm phải tách thành `Enrollment`? · `findByFullNameContainingIgnoreCase` sinh ra SQL gì? · `cascade = ALL` nguy hiểm ở đâu? · Vì sao nên tự tạo index cho cột khóa ngoại, index làm chậm thao tác nào? · Seq Scan nghĩa là gì, khi nào nó vẫn là lựa chọn đúng? Gắn tag `week-5`.
Xong khi	☐ Test repository chạy xanh ☐ Toàn bộ API tuần 3–5 chạy trên PostgreSQL ☐ Repo có tag `week-5`

Tuần 6 — Transaction, N+1, khóa dữ liệu, migration

30/11 – 04/12 · 5 buổi

Phần khó nhất của tháng, không cắt. Cuối tuần: đạt Tiêu chí hoàn thành.

Buổi 19 **Thứ Hai 30/11/2026** Phân trang và Flyway

Spring ≈ 1 giờ 50	• **15 phút** — Đọc tài liệu Flyway (Getting started, quy tắc đặt tên `V1__…sql`); Spring Data JPA reference → Paging and Sorting. • **40 phút** — `GET /api/students?page=&size=&sort=` dùng `Pageable`; map `Page<Student>` sang DTO trước khi trả (đừng trả entity); giới hạn `size` tối đa. • **55 phút** — Thêm Flyway. Chỉnh `schema.sql` cho khớp entity rồi đặt thành `db/migration/V1__init.sql` (có thể nhờ AI đối chiếu với entity; bạn review từng cột, khóa ngoại, index). Đổi `ddl-auto` sang **validate**, xóa database cũ, chạy lại từ đầu. Thêm `V2__add_indexes.sql` cho các index đã tạo ở tuần 5.
DB 50 phút	Phân trang `LIMIT/OFFSET` và phân trang theo khóa (`WHERE id > ? ORDER BY id LIMIT 20`); chi phí của `COUNT(*)`. So `OFFSET 50000` với cách theo khóa bằng `EXPLAIN ANALYZE`.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ App khởi động với Flyway và ddl-auto=validate ☐ API danh sách có phân trang, sắp xếp

Buổi 20 Thứ Ba 01/12/2026 @Transactional

Spring
≈ 1 giờ 55

- **Hôm nay làm phần DB trước** để thấy transaction ở mức SQL, rồi mới sang Spring.
- **20 phút** — Đọc Spring Framework reference → Data Access → Transaction Management → Declarative Transaction Management.
- **70 phút** — `PUT /api/students/{id}/class/{classId}`: chuyển lớp và ghi một dòng vào bảng lịch sử chuyển lớp (thêm migration `V3__class_transfer_history.sql`). Cố tình ném exception sau khi đã cập nhật sinh viên: không có `@Transactional` thì dữ liệu dở dang; có thì rollback toàn bộ. Đặt `@Transactional` ở tầng service; dùng `readOnly = true` cho các hàm chỉ đọc.
- **25 phút** — Tái hiện các trường hợp **không rollback**: ném checked exception; gọi hàm `@Transactional` từ một hàm khác trong cùng class. Ghi kết quả và lý do.

DB
50 phút

`BEGIN`, `COMMIT`, `ROLLBACK`, `SAVEPOINT`; ACID. Tắt auto-commit trong DBeaver, chuyển lớp bằng 2 câu `UPDATE`, rollback giữa chừng; mở cửa sổ thứ hai xem nó thấy gì trước và sau khi commit.
Đọc: PostgreSQL Tutorial → Transactions.

Cuối buổi
10–15 phút

Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.

Xong khi

- ☐ Lỗi giữa chừng khi chuyển lớp → không có thay đổi nào được lưu
- ☐ Tái hiện được ít nhất 2 trường hợp không rollback

Buổi 21 Thứ Tư 02/12/2026 Lỗi N+1

Spring
≈ 1 giờ 45

- **15 phút** — Vlad Mihalcea: đọc bài về N+1 query problem.
- **30 phút** — **Tạo lỗi**: `GET /api/classes` trả kèm danh sách sinh viên mỗi lớp (quan hệ LAZY). Đếm số câu SQL trong log với 10 lớp.
- **45 phút** — Sửa bằng `JOIN FETCH` trong `@Query`, rồi thử bằng `@EntityGraph`; so số câu SQL. Thử `JOIN FETCH` kèm `Pageable`, đọc cảnh báo của Hibernate về phân trang trong bộ nhớ và ghi lại.
- **15 phút** — Nhờ AI viết một endpoint có quan hệ, kiểm tra xem có dính N+1 không — ghi nhật ký.

DB
50 phút

So 1.000 câu `SELECT` nhỏ với 1 câu `JOIN` trả cùng dữ liệu: thời gian chạy và `EXPLAIN ANALYZE`.

Cuối buổi
10–15 phút

Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.

Xong khi

- ☐ Số câu SQL giảm từ N+1 xuống 1
- ☐ Ghi lại được vấn đề của JOIN FETCH khi phân trang

Buổi 22 **Thứ Năm 03/12/2026** Ghi đồng thời: optimistic locking

Spring ≈ 1 giờ 40	• **Hôm nay cũng làm phần DB trước**: tái hiện lỗi bằng 2 cửa sổ DBeaver rồi mới sang Spring. • **30 phút** — Lớp có `capacity` và `enrolled_count` (migration V4). `POST /api/classes/{id}/students` thêm sinh viên vào lớp, hết chỗ trả 409. • **30 phút** — **Tái hiện lỗi**: lớp còn 1 chỗ, gửi 2 request cùng lúc (test dùng `ExecutorService` + `CountDownLatch`, hoặc 2 lệnh `curl` chạy song song) → lớp vượt sĩ số. • **40 phút** — Sửa bằng `@Version` trên `ClassRoom`; bắt `ObjectOptimisticLockingFailureException` → 409. Chạy lại: chỉ 1 request thành công. (Tùy chọn) thử `@Lock(PESSIMISTIC_WRITE)` để so sánh.
DB 55 phút	Isolation level: PostgreSQL mặc định Read Committed; Repeatable Read; Serializable. Lost update; `SELECT … FOR UPDATE`; deadlock. 2 cửa sổ DBeaver cùng ghi danh vào lớp còn 1 chỗ: tái hiện lost update, sửa bằng `FOR UPDATE`. Đọc: PostgreSQL docs → Transaction Isolation, Explicit Locking.
Cuối buổi 10–15 phút	Tự giải thích 1 khái niệm của buổi bằng lời; ghi lỗi gặp phải và lỗi AI (nếu có) vào nhật ký; commit.
Xong khi	☐ 2 request cùng lúc: chỉ 1 thành công, request kia nhận 409 ☐ Tái hiện và sửa được lost update bằng SQL thuần

Buổi 23 **Thứ Sáu 04/12/2026** Tổng hợp và tự kiểm tra

Spring 2 giờ	• **60 phút** — Chạy lại toàn bộ từ database trống: `docker compose up` → app (Flyway tạo bảng) → Postman collection → `mvn test` xanh. Sửa mọi chỗ hỏng. • **30 phút** — Hoàn thiện `README.md`: cách chạy, danh sách API, ERD. • **30 phút** — Đối chiếu "Tiêu chí hoàn thành"; mục nào chưa đạt thì ghi lại để bù.
DB 45 phút	Tự thiết kế 4–5 bảng cho đề tài dự án (ví dụ đặt lịch khám) — chuẩn bị cho tuần 7. Nếu còn thời gian (hoặc cuối tuần): 5 bài LeetCode SQL 50 mức Medium.
Cuối buổi 30 phút	**30 phút tự kiểm tra tuần 6**: `@Transactional` không rollback khi nào? · N+1 xảy ra khi nào, `JOIN FETCH` và `@EntityGraph` khác nhau gì? · Optimistic và pessimistic locking khác nhau thế nào, xung đột thì trả status gì? · Vì sao dùng `ddl-auto=validate` khi đã có Flyway? · Sửa một file migration đã chạy thì chuyện gì xảy ra? · Read Committed cho phép hiện tượng gì mà Repeatable Read chặn được? · Bài lớp còn 1 chỗ nên chọn `FOR UPDATE` hay `@Version`, vì sao? Gắn tag `week-6`.
Xong khi	☐ Toàn bộ chạy được từ database trống bằng một lần docker compose up ☐ Đạt các mục trong Tiêu chí hoàn thành ☐ Repo có tag `week-6`

Tiêu chí hoàn thành (= điều kiện sang giai đoạn dự án)

☐ Giải thích được IoC/DI và vòng đời của một request trong Spring

☐ **Không dùng AI**, tự viết được API CRUD có DTO, validation và format lỗi chuẩn trong một buổi

☐ Giải thích được lỗi N+1 và hai cách sửa; biết `@Transactional` đặt ở đâu và khi nào nó không rollback

☐ Viết được câu SQL có JOIN, GROUP BY, subquery; đọc được `EXPLAIN` ở mức cơ bản; tự thiết kế được 4–5 bảng có quan hệ

☐ Dùng SQL thuần (không qua Spring) mô phỏng được một transaction rollback và một lần ghi đồng thời bị lost update, rồi sửa bằng `FOR UPDATE`

☐ Nhật ký review AI có ít nhất 10 lỗi đã bắt được

☐ Student Management API chạy với PostgreSQL và Flyway, đã có trên GitHub

Khi bị chậm

- **Chậm dưới 3 buổi:** bù vào thứ Bảy/Chủ nhật nếu rảnh; nếu không, rút gọn phần "AI viết, bạn review" của tuần sau (MapStruct, logging, profile, Flyway).
- **Phần DB bị chậm:** bỏ các bài LeetCode trước; giữ JOIN, ràng buộc, transaction và `FOR UPDATE`.
- **Không cắt** tuần 3 (tự gõ tay) và các buổi transaction, N+1, khóa dữ liệu của tuần 6.
- **Chưa đạt tiêu chí hoàn thành vào 04/12:** thêm tối đa 3–5 buổi (lấn sang tuần 7), rồi vẫn sang dự án và bù dần; phần dự phòng 18/01 – 04/02 vẫn còn.

Tài liệu tham khảo

Chủ đề	Tài liệu
HTTP	MDN: HTTP
Spring	spring.io/guides; Spring Framework reference; Spring Boot reference; Baeldung (lưu ý nhiều bài viết cho Spring Boot 3)
JPA / Database	Spring Data JPA reference; Vlad Mihalcea
SQL / PostgreSQL	PostgreSQL Tutorial; Use The Index, Luke; SQLBolt; LeetCode SQL 50
Phiên bản	Spring Boot wiki: Release Notes, Migration Guide